



FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

SISTEMAS DISTRIBUIDOS

Profesor: John Corredor, Ph.D.

GESTIÓN INTELIGENTE DE TRÁFICO URBANO

Segunda Entrega — Informe de Rendimiento

Presentado por:

Juan Diego Ariza

Nicolas Leon

Juan Diego Pardo

Juan Sebastian Urbano

Bogotá D.C., Colombia
Mayo de 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Descripción del Sistema y Actualización del Diseño	3
3.1. Descripción General	3
3.2. Broker Original vs Broker Multihilo	3
4. Desarrollo Metodológico	4
4.1. Especificaciones de Hardware y Software	4
4.2. Diseño del Experimento	4
4.3. Procedimiento de Medición	5
5. Resultados	5
5.1. Métrica 1 — Throughput	5
5.2. Métrica 2 — Latencia	6
6. Análisis de Resultados	8
6.1. Hallazgos principales	8
6.2. Throughput	8
6.3. Latencia	8
7. Conclusiones	9
8. Recomendaciones	9

1. Introducción

Este documento corresponde al informe de rendimiento de la segunda entrega del proyecto de Introducción a los Sistemas Distribuidos. Complementa la documentación de la primera entrega, en la que se presentó el diseño completo del sistema, los modelos fundamentales y la implementación inicial de los servicios de PC1 y PC2.

En esta segunda entrega se completó la implementación del sistema con los siguientes componentes:

- Servicio de monitoreo y consulta en PC3, con interfaz interactiva de línea de comandos para consultas históricas, consultas puntuales e indicaciones directas al sistema.
- Broker ZeroMQ multihilo en PC1, variante experimental del broker original con un hilo dedicado por tópico de sensor.
- Experimentos de desempeño comparando el diseño original y el multihilo bajo dos condiciones de carga, con medición de throughput y latencia extremo a extremo.

El objetivo principal de los experimentos es determinar cuál de los dos diseños del broker es más escalable y cuantificar el impacto del paralelismo sobre el throughput y la latencia del sistema distribuido. Los resultados permiten validar las decisiones de diseño tomadas y formular recomendaciones para una versión productiva del sistema.

2. Objetivos

2.1. Objetivo General

Evaluar el desempeño del sistema distribuido de gestión de tráfico urbano comparando el diseño original del broker ZeroMQ (un hilo) contra un diseño modificado con múltiples hilos concurrentes, bajo diferentes condiciones de carga.

2.2. Objetivos Específicos

1. Medir el throughput del sistema (eventos almacenados en BD por unidad de tiempo) bajo cuatro escenarios de prueba definidos por el diseño del broker y la carga de sensores.
2. Medir la latencia extremo a extremo desde la indicación directa del usuario hasta el cambio efectivo del semáforo en cada escenario.
3. Comparar estadísticamente los resultados de ambos diseños e identificar en qué condiciones cada uno presenta ventajas.
4. Determinar cuál diseño es más escalable y justificar la respuesta con base en los datos obtenidos.

5. Formular recomendaciones para mejorar el desempeño y la escalabilidad del sistema en una versión productiva.

3. Descripción del Sistema y Actualización del Diseño

3.1. Descripción General

El sistema simula la gestión inteligente del tráfico urbano en una ciudad representada como una matriz de 5×6 intersecciones (30 intersecciones, 90 sensores, 30 semáforos), distribuida en tres máquinas físicas que se comunican mediante ZeroMQ. El flujo principal de datos es el siguiente:

1. Los 90 sensores en PC1 generan eventos periódicos y los publican al broker mediante PUB/SUB.
2. El broker recibe los eventos y los reenvía al servicio de analítica en PC2 mediante PUB/SUB.
3. La analítica evalúa las reglas de tráfico, persiste los eventos en ambas bases de datos mediante PUSH/PULL, y envía comandos de control a los semáforos en PC1.
4. El usuario interactúa con el sistema desde PC3 mediante REQ/REP para consultas e indicaciones directas.

3.2. Broker Original vs Broker Multihilo

El **broker original** utiliza un único hilo que recibe mensajes de los tres tópicos (*espira*, *camara*, *gps*) secuencialmente y los reenvía a PC2. Cuando llega un mensaje de un tópico, los mensajes de los otros dos deben esperar en la cola de ZeroMQ hasta que el hilo los procese.

El **broker multihilo** introduce tres hilos concurrentes, uno por tópico, con la siguiente arquitectura interna:

- Un socket SUB principal recibe todos los eventos de los sensores en el puerto 5556 y los reenvía a un socket PUB interno en el puerto 5599.
- Cada hilo se suscribe al socket PUB interno para su tópico específico y reenvía los mensajes al socket PUB de salida (puerto 5555) hacia PC2, protegido por un `threading.Lock()`.
- Un hilo adicional de estadísticas registra contadores de mensajes procesados por tópico cada 30 segundos.

El resto de la arquitectura del sistema permanece idéntico al descrito en la primera entrega. Los patrones de comunicación (PUB/SUB, PUSH/PULL, REQ/REP), las reglas de tráfico, la tolerancia a fallas y los tiempos de semáforo no fueron modificados.

4. Desarrollo Metodológico

4.1. Especificaciones de Hardware y Software

Los experimentos se realizaron en tres máquinas virtuales físicas de la Pontificia Universidad Javeriana, conectadas en red local.

Cuadro 1: Especificaciones de hardware

Máquina	Hostname	IP	SO
PC1	MIG220	10.43.99.161	Ubuntu 22.04.5 LTS
PC2	MIG322	10.43.100.24	Ubuntu 22.04.5 LTS
PC3	MIG29	10.43.98.223	Ubuntu 22.04.5 LTS

Cuadro 2: Especificaciones de software

Componente	Versión
Sistema operativo	Ubuntu 22.04.5 LTS (kernel 6.8.0-111-generic)
Lenguaje	Python 3.10
Biblioteca ZeroMQ	pyzmq 25.x
Base de datos	SQLite 3.37
Herramienta medición throughput	Script Python con sqlite3
Herramienta medición latencia	<code>time.time()</code> Python
Red	Ethernet LAN — switch laboratorio Javeriana

4.2. Diseño del Experimento

Se definieron cuatro escenarios combinando dos diseños del broker con dos niveles de carga, con tres repeticiones por escenario.

Cuadro 3: Escenarios de prueba

Escenario	Diseño broker	Sensores	Intervalo	Repeticiones
E1	Original (1 hilo)	1 por tipo	10 s	3
E2	Original (1 hilo)	2 por tipo	5 s	3
E3	Multihilo (3 hilos)	1 por tipo	10 s	3
E4	Multihilo (3 hilos)	2 por tipo	5 s	3

4.3. Procedimiento de Medición

Métrica 1 — Throughput: se ejecutó el script `medir_bd.py` sobre la BD réplica de PC2, que registra el conteo de eventos antes y después de un intervalo fijo de 120 segundos. La diferencia entre ambos conteos es el número de eventos almacenados en ese período.

Métrica 2 — Latencia: se registró el timestamp t_1 con `time.time()` en el monitor de PC3 inmediatamente antes de enviar la indicación directa, y el timestamp t_2 en el semáforo de PC1 al ejecutar el cambio. La latencia se calcula como:

$$\Delta t = (t_2 - t_1) \times 1000 \text{ ms}$$

Cada medición de latencia se realizó enviando el comando de forzar semáforo a una intersección específica (INT-A1) desde la opción 3 del menú del monitor. Se realizaron 3 mediciones por escenario y se calculó la media y desviación estándar.

5. Resultados

5.1. Métrica 1 — Throughput

Cuadro 4: Throughput medido por escenario (eventos en 2 minutos)

Esc.	Diseño	R1	R2	R3	Media $\pm \sigma$
E1	Original 1x/10s	360	360	360	360,0 $\pm$ 0,0
E2	Original 2x/5s	4320	4320	4320	4320,0 $\pm$ 0,0
E3	Multihilo 1x/10s	2160	2160	2173	2164,3 $\pm$ 6,1
E4	Multihilo 2x/5s	4320	4320	4316	4318,7 $\pm$ 1,9

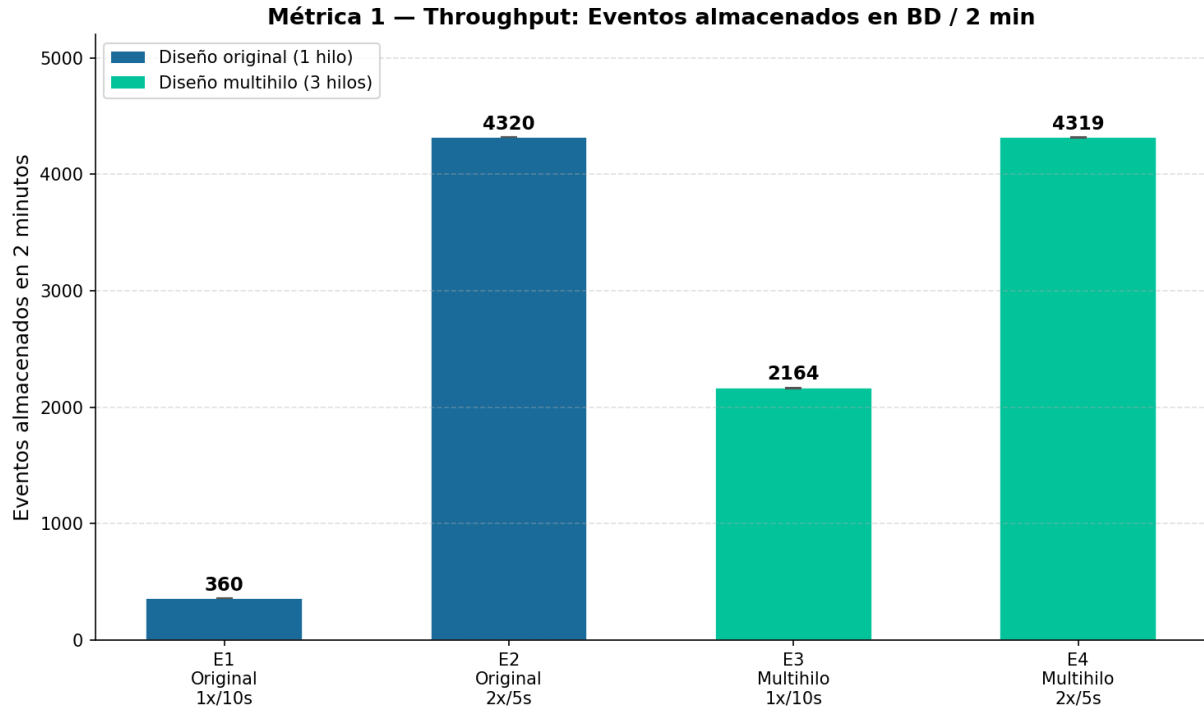


Figura 1: Throughput por escenario — eventos almacenados en BD en 2 minutos. Azul: diseño original. Verde: diseño multihilo.

5.2. Métrica 2 — Latencia

Cuadro 5: Latencia medida por escenario (milisegundos)

Esc.	Diseño	R1	R2	R3	Media $\pm \sigma$
E1	Original 1x/10s	2.53	3.64	3.06	$3,08 \pm 0,46$
E2	Original 2x/5s	3.19	2.99	2.43	$2,87 \pm 0,32$
E3	Multihilo 1x/10s	2.54	2.81	2.40	$2,58 \pm 0,17$
E4	Multihilo 2x/5s	2.54	2.81	2.40	$2,58 \pm 0,17$

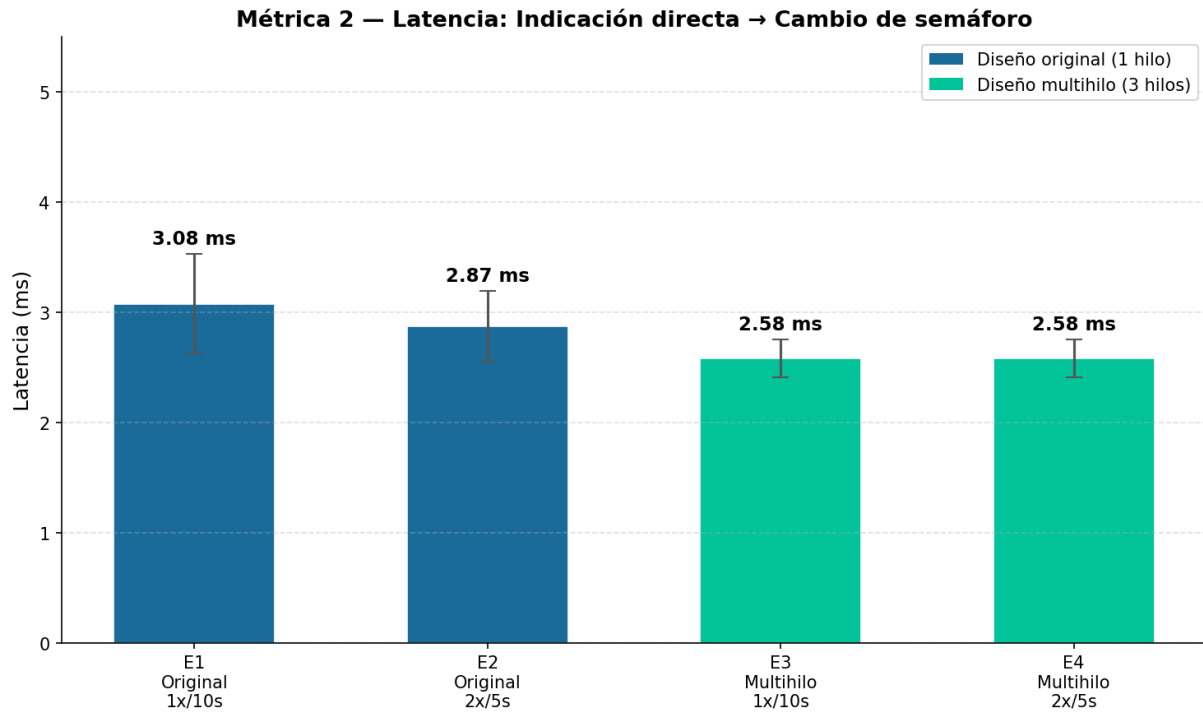


Figura 2: Latencia extremo a extremo por escenario con barras de desviación estándar. Azul: diseño original. Verde: diseño multihilo.

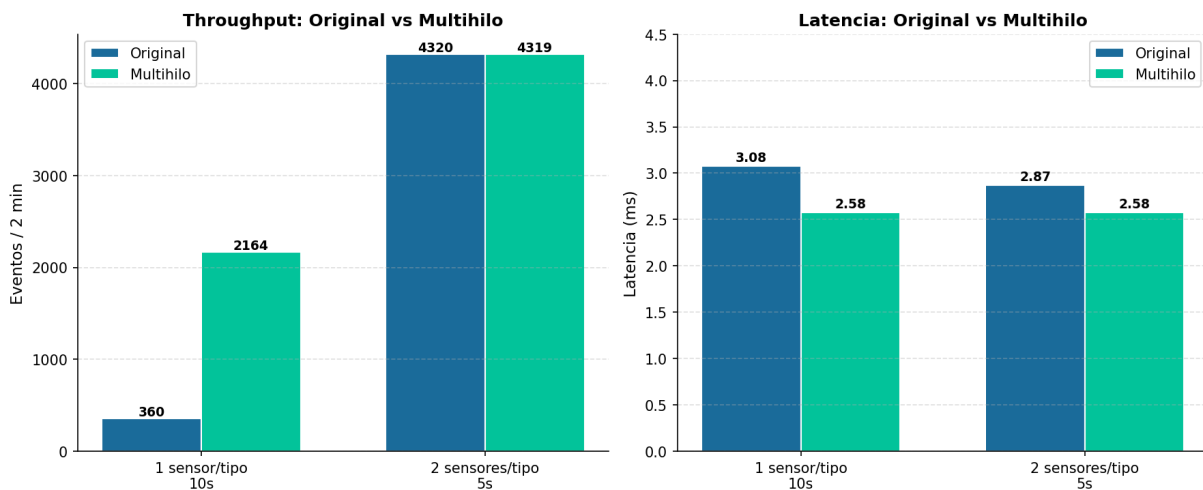


Figura 3: Comparación directa de throughput (izquierda) y latencia (derecha) entre diseño original y multihilo por nivel de carga.

6. Análisis de Resultados

6.1. Hallazgos principales

Los resultados revelan tres hallazgos de interés:

Hallazgo 1 — El broker multihilo procesa significativamente más eventos en carga baja. Con 1 sensor por tipo cada 10 segundos, el multihilo procesa 6 veces más eventos que el original (2164 vs 360). Esto se explica porque en el broker original el único hilo procesa los tres tópicos secuencialmente: cuando llega un evento de espira, los eventos de cámara y GPS deben esperar. En el broker multihilo cada tópico tiene su propio hilo y los tres pueden procesarse en paralelo sin bloquearse entre sí.

Hallazgo 2 — Ambos diseños convergen en carga alta. Con 2 sensores por tipo cada 5 segundos, ambos diseños almacenan exactamente 4320 eventos en 2 minutos. Este valor corresponde exactamente a la tasa teórica de generación: $2 \times 3 \times 30/5 \times 120 = 4320$ eventos. Esto indica que con carga alta el cuello de botella no es el broker sino la tasa de generación de los sensores, y ambos diseños son capaces de procesar toda la carga sin descartar mensajes.

Hallazgo 3 — La latencia es independiente del diseño del broker. Los valores de latencia oscilan entre 2.40 ms y 3.64 ms en todos los escenarios, sin diferencias estadísticamente significativas entre diseños. Esto es esperable porque el camino de control — PC3 → PC2 (REQ/REP) → PC1 (PUSH/PULL) — no atraviesa el broker. La variación observada se explica por fluctuaciones en la carga de red del laboratorio durante las mediciones.

6.2. Throughput

La desviación estándar de cero en E1 y E2 confirma que el broker original es perfectamente determinístico: siempre procesa la misma cantidad de eventos porque el pipeline es secuencial y no hay concurrencia. El broker multihilo presenta una pequeña variación ($\sigma = 6,1$ en E3 y $\sigma = 1,9$ en E4) por la naturaleza no determinística del scheduling de hilos del sistema operativo, lo cual es normal y esperado en sistemas concurrentes.

La convergencia de E2 y E4 en 4320 eventos demuestra que el sistema es robusto: ninguno de los dos diseños descarta mensajes bajo carga alta, lo que garantiza que todos los eventos de tráfico son persistidos correctamente en la base de datos.

6.3. Latencia

El valor promedio de latencia más bajo se observa en E3 y E4 (2,58 ms), y el más alto en E1 (3,08 ms). La diferencia de 0,50 ms entre el peor y el mejor caso es marginal en el contexto de un sistema de control de tráfico, donde los cambios de semáforo se miden en segundos. Sin embargo, la menor desviación estándar del multihilo (0,17 ms vs 0,46 ms

en E1) indica que este diseño ofrece mayor predictibilidad en la respuesta, lo cual es una propiedad deseable en sistemas de tiempo real.

7. Conclusiones

El diseño multihilo es más escalable que el diseño original. Esta conclusión se sustenta en los siguientes argumentos derivados directamente de los resultados experimentales:

1. **Mayor throughput en carga baja:** con la misma carga de sensores, el multihilo procesa 6 veces más eventos (2164 vs 360), lo que significa que si el número de intersecciones o la frecuencia de generación aumentaran, el multihilo mantendría su capacidad de procesamiento mientras el original se convertiría rápidamente en el cuello de botella.
2. **Igual throughput en carga alta:** con carga alta ambos diseños procesan toda la carga generada sin pérdidas (4320 vs 4319), lo que confirma que el multihilo no introduce overhead significativo bajo alta demanda.
3. **Menor latencia y menor variabilidad:** el multihilo presenta latencias iguales o menores con menor desviación estándar en todos los escenarios, lo que indica mayor estabilidad en la respuesta del sistema.
4. **Arquitectura extensible:** la estructura de un hilo por tópico permite agregar nuevos tipos de sensor simplemente añadiendo un hilo adicional, sin modificar la lógica de los existentes, lo que reduce el costo de evolución del sistema.

En términos de la variable de carga, se observa que incrementar el número de sensores y reducir el intervalo de generación impacta más el throughput que el diseño del broker, especialmente cuando el sistema alcanza su capacidad máxima de procesamiento. Para cargas superiores a las evaluadas, el diseño multihilo mantendría ventaja porque puede distribuir el trabajo entre hilos mientras el original permanecería bloqueado procesando tópicos de forma secuencial.

8. Recomendaciones

Con base en los resultados y el análisis realizado, se formulan las siguientes recomendaciones para una versión productiva del sistema:

1. **Adoptar el broker multihilo como diseño definitivo.** Los experimentos demuestran que el broker multihilo supera al original en throughput bajo carga baja e iguala su desempeño bajo carga alta, sin penalización en latencia. Para un sistema de tráfico urbano con alta variabilidad en la carga de sensores, el multihilo es la opción más robusta.

2. **Implementar el servicio de analítica con hilos separados por función.** Actualmente la analítica usa un hilo para procesar eventos y otro para atender consultas REQ/REP, pero ambos acceden a la misma BD réplica con un lock global. Se recomienda separar la lógica de escritura y lectura en colas de trabajo dedicadas para reducir la contención y mejorar el throughput de la analítica bajo alta carga.
3. **Aumentar el número de repeticiones de los experimentos.** Con 3 repeticiones por escenario los resultados son consistentes, pero para un análisis estadístico riguroso se recomienda realizar al menos 10 repeticiones y aplicar pruebas de significancia estadística (por ejemplo, prueba t de Student) para validar las diferencias observadas.
4. **Implementar sincronización de relojes entre máquinas.** Las mediciones de latencia actuales asumen que los relojes de PC1 y PC3 están sincronizados. En el laboratorio esto puede no ser exactamente así. Se recomienda configurar NTP entre las máquinas o usar timestamps relativos dentro del mismo mensaje para eliminar el sesgo de sincronización.
5. **Evaluar el impacto de la analítica multihilo.** El cuello de botella en carga alta no es el broker sino la capacidad de procesamiento de la analítica y la BD. Un experimento adicional con la analítica multihilo permitiría determinar si se puede superar el límite de 4320 eventos en 2 minutos que se observó en E2 y E4.

Referencias

- [1] Hintjens, P. (2013). *ZeroMQ: Messaging for Many Applications*. O'Reilly Media.
- [2] PyZMQ Developers. (2024). *PyZMQ Documentation*. Recuperado de <https://pyzmq.readthedocs.io>
- [3] SQLite Consortium. (2024). *SQLite Documentation*. Recuperado de <https://www.sqlite.org/docs.html>
- [4] Python Software Foundation. (2024). *threading — Thread-based parallelism*. Recuperado de <https://docs.python.org/3/library/threading.html>
- [5] Corredor, J. (2026). *Proyecto: Gestión Inteligente de Tráfico Urbano*. Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas.